THE LEARNING BISTRO

Operations, Reservations & AI Tool Knowledge

Purpose: Learning document for an AI-powered restaurant management system.

---

1. Restaurant AI Assistant

The Learning Bistro uses an AI assistant to help customers interact with the restaurant.

The AI assistant can answer questions about:

- Restaurant timings
- Restaurant facilities
- Menu
- Menu prices
- Dietary information
- Reservation policies
- Table availability
- Reservations
- Reservation cancellation
- Existing reservations

The assistant uses different tools depending on the customer's request.

---

2. Important Data Separation

The restaurant application has two major types of information.

RAG Knowledge

The RAG system contains relatively static restaurant information.

Examples include:

- Restaurant information
- Menu
- Menu prices

- Opening hours
- Facilities
- Reservation policies
- Cancellation policies
- FAQs

This information is stored in PDF documents and indexed in a vector database.

Cassandra Data

Cassandra stores dynamic transactional information.

Examples include:

- Customers
- Tables
- Reservations
- Reservation status
- Reservation date
- Reservation time
- Number of guests
- Table assignments

The AI assistant should use Cassandra-backed tools whenever it needs current reservation information.

---

3. AI Tools

The restaurant agent has access to the following conceptual tools.

Tool 1 — search_menu

Purpose: Search restaurant menu information.

The tool can retrieve:

- Menu items
- Prices
- Ingredients
- Categories

- Vegetarian information
- Dietary information
- Descriptions

Data Source: Restaurant RAG / Vector Database

Example

Customer:

"How much is Butter Chicken?"

Agent:

```
search_menu("Butter Chicken")
```

---

4. search_restaurant_info

Purpose: Search general restaurant information.

The tool can retrieve:

- Opening hours
- Closing hours
- Restaurant facilities
- Seating capacity
- Reservation rules
- Cancellation policy
- Restaurant FAQs

Data Source: Restaurant RAG / Vector Database

Example

Customer:

"What time do you close on Sunday?"

Agent:

```
search_restaurant_info("Sunday closing time")
```

---

5. check_availability

Purpose: Determine whether a suitable table is available.

Data Source: Cassandra

The tool should consider:

- Requested date
- Requested time
- Number of guests
- Existing reservations
- Table capacity
- Table status

Example

Customer:

"Do you have a table for four tonight at 8 PM?"

Agent:

```
check_availability(date, time, guests=4)
```

The tool returns the available table or tables.

---

6. create_reservation

Purpose: Create a new restaurant reservation.

Data Source: Cassandra

The tool should receive information such as:

- Customer ID
- Customer name
- Date
- Time
- Number of guests
- Table ID

Example

Customer:

"Book the table for me."

Agent:

```
create_reservation(...)
```

The system stores the reservation in Cassandra and returns a reservation ID.

---

7. get_reservation

Purpose: Retrieve an existing reservation.

Data Source: Cassandra

The tool can retrieve:

- Reservation ID
- Customer
- Date
- Time
- Number of guests
- Table
- Reservation status

Example

Customer:

"What time is my reservation?"

Agent:

```
get_reservation(reservation_id)
```

---

8. cancel_reservation

Purpose: Cancel an existing reservation.

Data Source: Cassandra

Example

Customer:

"Cancel my reservation RES-1001."

Agent:

```
cancel_reservation("RES-1001")
```

The reservation status is updated in Cassandra.

---

9. Example Agent Flow — Menu Question

Customer:

"What vegetarian starters do you have?"

Agent Flow

```
User
  |
  v
AI Agent
  |
  | decides that menu information is required
  v
search_menu
  |
  v
Vector Database
  |
  v
Relevant menu chunks
  |
  v
AI Agent
  |
  v
Answer to User
```

The agent does not need Cassandra for this request.

---

10. Example Agent Flow — Restaurant Timing

Customer:

"What time does the restaurant close on Sunday?"

Agent Flow

```
User
  |
  v
AI Agent
  |
  v
search_restaurant_info
  |
  v
Vector Database
  |
  v
Restaurant Information PDF
  |
  v
AI Agent
  |
  v
Answer
```

---

11. Example Agent Flow — Availability

Customer:

"Is a table for four available tonight at 8 PM?"

Agent Flow

```
User
  |
  v
AI Agent
  |
  v
check_availability
```

```
  |
  v
Cassandra
  |
  +---- Tables
  |
  +---- Existing Reservations
  |
  v
Availability Result
  |
  v
AI Agent
  |
  v
Answer
```

The RAG system should not be used to determine live availability.

---

12. Example Agent Flow — Complete Booking

Customer:

"Book a table for four tonight at 8 PM."

The agent should coordinate multiple tools.

```
User
 |
 v
AI Agent
 |
 v
check_availability
 |
 v
Cassandra
 |
```

```
 v
Available?
 |
 +---- NO ----> Agent tells user no table is available
 |
 |
 YES
 |
 v
create_reservation
 |
 v
Cassandra
 |
 v
Reservation Created
 |
 v
AI Agent
 |
 v
Reservation Confirmation
```

This is an important example of why an agent is useful.

The agent decides that it needs to:

1. Check availability.
2. Examine the result.
3. If available, create the reservation.
4. Return the result to the customer.

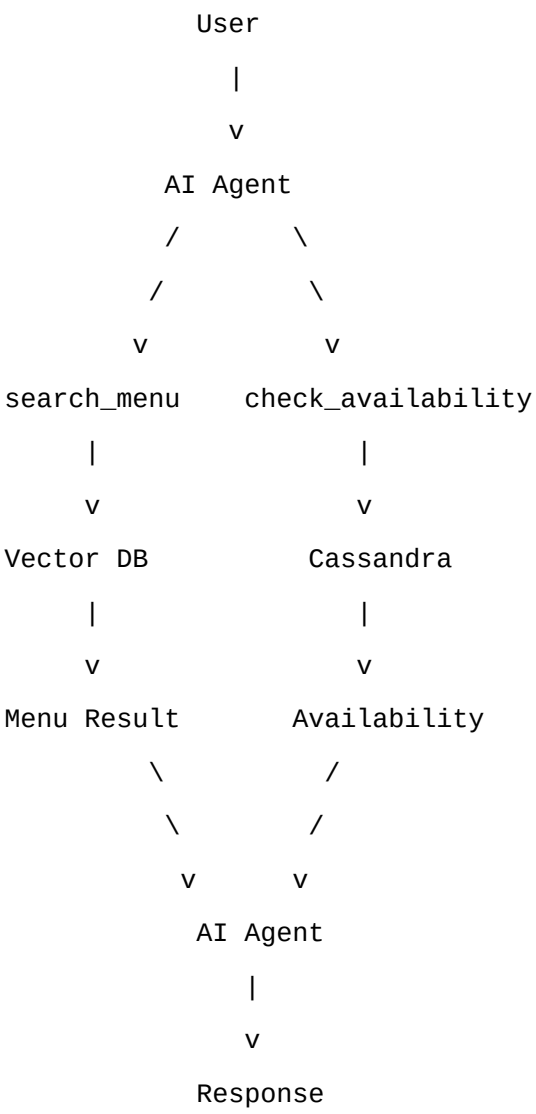
---

13. Example Flow — Menu + Reservation

Customer:

"Do you have vegetarian food and can I book a table for two tonight at 7 PM?"

The agent may need multiple tools.



The agent should decide which tools are necessary rather than blindly calling every tool.

---

14. Example RAG Questions

The following questions should normally use the RAG tools:

- What are your opening hours?
- What time do you close on Sunday?
- What vegetarian dishes do you have?
- How much is Butter Chicken?
- Do you have Chicken Biryani?
- What desserts do you serve?

- Do you have outdoor seating?
- What is your cancellation policy?
- How many people can the restaurant accommodate?
- Do you have wheelchair-accessible seating?

---

15. Example Cassandra Questions

The following questions require live database information:

- Is a table available at 8 PM?
- How many tables are free?
- What is my reservation?
- Cancel my reservation.
- Book a table for four.
- What table was assigned to my reservation?
- What is the status of reservation RES-1001?

---

16. AI Agent Rules

The AI agent should follow these rules.

Rule 1

Never invent menu information.

Rule 2

Never invent restaurant opening hours.

Rule 3

Never claim a table is available without checking the live reservation system.

Rule 4

Never create a reservation without checking availability first.

Rule 5

Never claim that a reservation was successfully created unless the reservation tool confirms it.

Rule 6

Use the RAG system for restaurant knowledge.

Rule 7

Use Cassandra-backed tools for live reservation information.

Rule 8

The agent should select tools based on the user's request.

---

17. Technology Stack

The learning project uses the following technologies:

Frontend

ReactJS

Backend

Python
FastAPI

AI Framework

LangChain

LLM

Ollama
Qwen3:30B

Transactional Database

Apache Cassandra

RAG

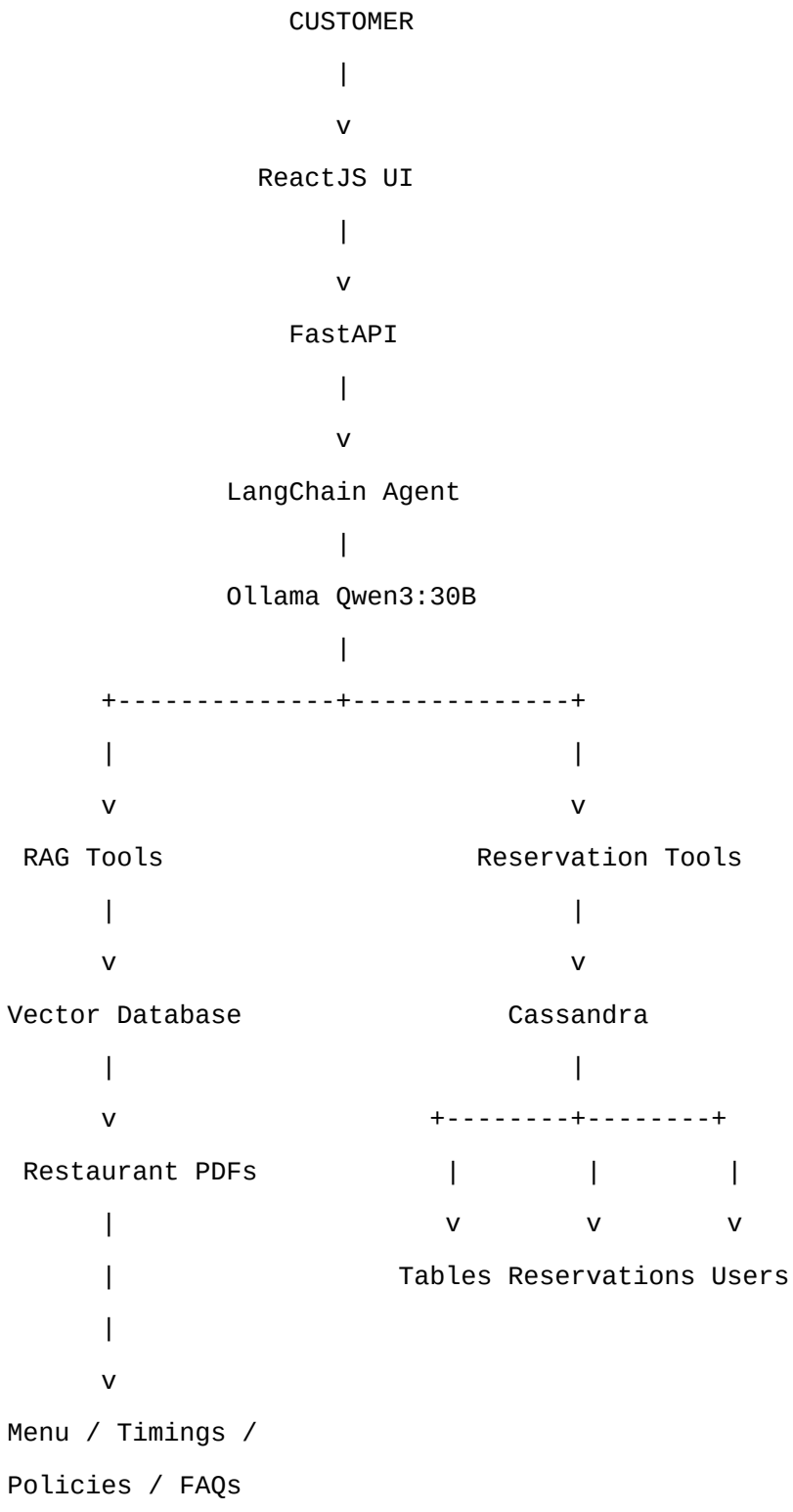
PDF documents
Document loader
Text splitter
Embeddings
Vector database

Agent

LangChain Agent

---

18. Example Complete Architecture



---

19. Key Principle

The restaurant's static knowledge and dynamic operational data should remain separate.

PDF + RAG answers:

> "What is on your menu?"

Cassandra answers:

> "Is table 7 available at 8 PM?"

Agent coordinates both:

> "Is there a table for four at 8 PM, and what vegetarian dishes can we order?"

The agent determines which tools are necessary, calls them, evaluates their results, and produces the final response.